Laurentian University
Faculty of Science, Engineering and Architecture
School of Engineering and Computer Science

Imbalance-Aware Attention for Network Intrusion Detection

A negative result on CICIDS-2017

Kritish Dhungel
MSc Computational Sciences
Project in Computational Science

Supervisor
Dr. Kalpdrum Passi

Reviewer
Dr. SK Md Mizanur Rahman

August 2026

Abstract

This project set out to test whether making a Transformer's attention mechanism aware of class frequency would improve detection of rare attack classes on CICIDS-2017, a dataset where BENIGN traffic accounts for 80.3% of flows and the rarest attack class has eleven samples. A class frequency bias term, alpha multiplied by log of one over the class frequency, was added to the model and evaluated against six baselines.

The hypothesis does not hold. An ablation with alpha fixed at zero, run across three seeds on an identical code path, matched or outperformed the full model on every seed. An apparent architectural advantage of 0.122 macro F1 over PyTorch's standard encoder was traced to a missing attention dropout call rather than to design. The headline result of an earlier draft, XSS recall rising from 0.58 to 0.95, does not survive the inclusion of precision: XSS F1 falls from 0.165 to 0.144, and the gain is absorbed from Web Attack Brute Force, whose F1 falls from 0.280 to 0.169.

Two further findings emerged that were not anticipated. Random Forest with SMOTE reaches macro F1 0.8985 across three seeds while the best Transformer reaches 0.6315, and trains roughly twenty-three times faster. A 1D-CNN reaches 0.8216 and is statistically indistinguishable from Random Forest, so the deficit is specific to the Transformer rather than general to deep learning.

Separately, replacing the random split with a day-based one collapses binary performance from macro F1 0.998 to between 0.45 and 0.53, with attack recall falling from above 0.999 to below 0.09. Thirteen percent of test flows under the random split are byte-identical to a training flow, rising to 58.8% for PortScan. Reported performance on this dataset is substantially an artefact of how it is partitioned.

Keywords: intrusion detection, class imbalance, Transformer, attention mechanism, CICIDS-2017, logit adjustment, gradient boosting, negative result, ablation study

Acknowledgements

My thanks go first to my supervisor, Dr. Kalpdrum Passi, who made this project possible in every practical sense. He provided access to the Digital Research Alliance of Canada through his own allocation, without which none of the sixty-odd training runs reported here could have been carried out. He arranged the external review that reshaped the work, and when the original hypothesis did not hold he redirected the project toward the experiments that produced its most useful results, including the binary formulation and the removal of the least measurable class. His guidance at the point where the work had stalled is the reason this report contains findings rather than an inconclusive negative.

I am also grateful to Dr. SK Md Mizanur Rahman, who reviewed an earlier draft and returned nineteen detailed technical comments. Several of the central findings exist because of that review. The observation that a scalar bias added inside softmax cannot alter the attention weights identified the defect in the original implementation. The requirement for a true ablation with alpha fixed at zero on an identical code path produced the result that falsified the central hypothesis. The prediction that any apparent gain in XSS recall would be absorbed from Web Attack Brute Force proved correct, and following it revealed that the headline result of the earlier draft did not survive the inclusion of precision. The report is considerably stronger in every respect that the review reached.

Computational resources were provided by the Digital Research Alliance of Canada through the Narval cluster at Calcul Quebec, under allocation def-kpassi held by Dr. Passi.

Table of Contents

1. Problem Statement

Every intrusion detection dataset before CICIDS-2017 had significant flaws. KDD99 contained 78% duplicate records, which made model results inconsistent and unreliable. CICIDS-2017 was introduced by the Canadian Institute for Cybersecurity in 2017 with eleven criteria defining what a valid intrusion detection dataset should contain. No existing dataset met those criteria, so the researchers generated their own by simulating attack and defence scenarios in a controlled lab environment alongside traffic designed to mimic real human behaviour.

CICIDS-2017 nonetheless has a severe class imbalance problem. BENIGN traffic accounts for 80.3% of the dataset, while the rarest attack classes have as few as eleven samples out of 2,830,743 network flows. A model can therefore attain 80% accuracy by predicting BENIGN for every input while detecting nothing. In a security setting this is a substantial failure, since a false negative constitutes a breach: malicious traffic labelled benign passes through undetected.

2. Dataset

CICIDS-2017 contains 2,830,743 network flow records across 79 features and 15 classes. It was captured over five working days in a controlled lab environment, with each day stored as a separate CSV file. Monday contains only benign traffic; Wednesday mixes DoS attacks with normal traffic; the remaining days each introduce different attack families. Features are network flow statistics extracted using CICFlowMeter, covering packet lengths, flow durations, inter-arrival times, TCP flag counts and flow rate metrics.

Table 1. Class distribution in the full dataset.

Class	Samples	Proportion
BENIGN	2,273,097	80.30%
DoS Hulk	231,073	8.16%
PortScan	158,930	5.61%
DDoS	128,027	4.52%
DoS GoldenEye	10,293	0.36%
FTP-Patator	7,938	0.28%
SSH-Patator	5,897	0.21%
DoS slowloris	5,796	0.20%
DoS Slowhttptest	5,499	0.19%
Bot	1,966	0.07%
Web Attack - Brute Force	1,507	0.05%
Web Attack - XSS	652	0.02%
Infiltration	**36**	**0.001%**
Web Attack - SQL Injection	**21**	**0.0007%**
Heartbleed	**11**	**0.0004%**

The three classes in bold are the ones this project set out to improve. Their sample counts are the principal difficulty. With eleven, twenty-one and thirty-six samples respectively they are difficult to learn and, as the results below demonstrate, impossible to evaluate reliably.

3. Preprocessing

All eight daily CSV files were merged. Column names were stripped of whitespace, and 1,358 missing values and 4,376 infinite values were removed, arising mostly from rate-based features computed on zero-duration flows.

Negative Flow Duration values, with a minimum of -13 microseconds, were checked before removal to see whether they belonged to rare attack classes. Removing rows from classes that are already scarce would have worsened the imbalance. All 75 rows were found to belong to BENIGN and were removed. Seven sparse TCP flag columns, namely Fwd and Bwd PSH Flags, Fwd and Bwd URG Flags, RST Flag Count, CWE Flag Count and ECE Flag Count, contained near-zero values almost exclusively in BENIGN samples and were dropped. An ablation retaining these columns is reported in Section 9.5.

Table 2. Data flow accounting, rows in and rows out.

Stage	Rows	Running total
Merged from eight daily CSV files	2,830,743	2,830,743
Removed: rows containing NaN or infinite values	-2,867	2,827,876
Removed: rows with negative Flow Duration	-75	2,827,801
Retained for modelling		**2,827,801**

The 2,867 removed rows contained 5,734 bad values between them, being the 1,358 missing and 4,376 infinite values noted above. That is exactly two per row. The likely explanation is that a zero-duration flow makes both Flow Bytes/s and Flow Packets/s undefined through the same division, so a single malformed flow produces two bad values in the same record. This has not been verified directly against the data and is offered as the most plausible reading of the arithmetic.

The split sizes are X_train 1,980,568, X_val 423,051 and X_test 424,182, which sum to the retained total.

The cleaned dataset has 2,827,801 rows and 71 features. A stratified 70/15/15 train, validation and test split was used for the main experiments, so each partition keeps the same class proportions. StandardScaler was fitted on the training partition only and applied to validation and test, which stops distributional information leaking across. StandardScaler was chosen over MinMaxScaler because network traffic contains extreme outliers from attack flows, and min-max scaling would compress normal traffic into a very narrow range.

SMOTE was applied to the training set only, raising each minority class to 10,000 samples. This threshold provides sufficient examples for the model to learn from without allowing synthetic data to dominate the training distribution. Classes already exceeding 10,000 samples were left unchanged. For the k-fold experiments SMOTE was applied inside each training fold separately, so no synthetic sample ever appeared in a validation fold.

Random Forest used the unscaled clean data with class_weight set to balanced. XGBoost used the same unscaled data with no imbalance handling at all. No SMOTE, no sample weighting, no scale_pos_weight. The significance of this distinction is discussed in Section 7.4.

4. Proposed Approach

Standard Transformer attention has no representation of class frequency. A Heartbleed flow is weighted identically to a BENIGN one, despite Heartbleed being roughly ten thousand times rarer. The hypothesis tested here was that making attention frequency-aware would improve rare-class detection.

Applying Transformers to tabular data has precedent in TabTransformer (Huang et al., 2020) and FT-Transformer (Gorishniy et al., 2021), both of which show attention can capture feature interactions in unordered tabular settings. This project follows that approach, treating each of the 71 flow features as a token.

$$\mathrm{IAA}(Q, K, V) = \mathrm{softmax}\left(QK^{\mathsf{T}} / \sqrt{d} + \alpha \cdot \log(1/f_c) \right) \cdot V$$

Here f_c is the frequency of class c in the training data and alpha is a learnable scaling parameter constrained positive. For rare classes f_c is small, so log of one over f_c is large. When alpha is zero the term vanishes and the model reduces to a standard Transformer, which provides the ablation baseline.

4.1 Two placements, and why the first failed

The bias was first added inside the attention score matrix, before softmax. This placement has no effect on the attention weights. The bias tensor had shape [batch, 1, 1, 1] and broadcast onto a score matrix of shape [batch, heads, 71, 71], so every key position within a row received the same value. Softmax normalises along that dimension and is invariant to a constant added across it. In index terms, score s_ij became s_ij + b where b depends only on the sample, not on i or j, so softmax over j is unchanged. Versions one through four were therefore inert. The attention weights were identical to a plain Transformer's.

The bias was subsequently moved to the output logits, where each of the fifteen class scores receives a different value and softmax cannot cancel it. This second form is well defined and is mathematically equivalent to logit adjustment (Menon et al., 2021) with a learnable rather than fixed scale.

One further defect in the early implementation should be recorded. The Dataset class returned the bias by looking it up from the sample's true label, and the evaluation loop passed that value into the model. At inference the true label is not available, so this is label leakage. It had no effect on any prediction, because those are the same versions in which softmax cancelled the bias, but the code path was wrong and the results from v1 to v4 should be read with that in mind. The corrected version reads no label at inference: a fixed fifteen-element vector computed once from training frequencies is added to every sample's logits regardless of its class.

Class-weighted cross entropy was also evaluated as a training-time complement. Both mechanisms are assessed in Sections 7 and 8. Neither improved performance, and the weighted loss degraded it substantially.

5. Related Work

Several prior works address class imbalance in long-tail classification. Menon et al. (2021) proposed logit adjustment, adding a label frequency-based bias to output logits at inference to encourage larger margins between rare and dominant class scores. Their evaluation was on image classification benchmarks. The logit-level formulation used here is equivalent to their method with a learnable scale, applied to tabular network traffic.

Cui et al. (2019) proposed class-balanced focal loss, weighting the training loss by class frequency so mistakes on rare classes are penalised more heavily, evaluated on object detection. A comparable weighting strategy was evaluated here alongside logit adjustment.

Ren et al. (2020) proposed Balanced Softmax, incorporating class priors directly into the softmax during training rather than as a post-hoc correction at inference.

None of these methods had previously been evaluated on network intrusion detection with CICIDS-2017, nor combined within a Transformer architecture for tabular security data. This project reports the outcome of that evaluation.

6. Models and Experimental Setup

Seven model configurations were trained and evaluated.

Random Forest served as the first classical baseline: an ensemble that handles high-dimensional data efficiently, trains quickly, and provides feature importance scores. XGBoost was the second, building trees sequentially so each corrects the errors of its predecessor, an approach that generally outperforms Random Forest on structured tabular data.

A 1D-CNN was the first deep baseline, applying convolutional filters across groups of features to detect local patterns. BiLSTM followed, processing the 71 features as a bidirectional sequence. It should be noted that network flow features have no intrinsic order and permuting the columns changes nothing semantically, so the sequential assumption is not well justified for this data. BiLSTM is retained for comparison rather than on principled grounds.

The standard Transformer, using nn.TransformerEncoderLayer with matched hyperparameters, was intended as the direct baseline for the IAA-Transformer. It was assumed any difference between the two could be attributed to the IAA modification. That assumption proved incorrect and is addressed in Section 7.3.

Both Transformer configurations used d_model 128, three encoder layers and twenty training epochs. An initial experiment at d_model 64 produced weak results, as 64 dimensions were insufficient to represent relationships across 71 features. Three layers outperformed two on minority classes. Twenty epochs were needed for convergence given a learning rate of 0.0005, chosen over 0.001 for more stable training.

7. Results

All results below are on the held-out validation set of 423,051 flows. F1 is reported throughout rather than recall alone, because recall without precision cannot distinguish a model that detects a rare class from one that over-predicts it. This distinction proves decisive in the results that follow.

7.1 Per-class comparison across all models

Table 3. F1 by class for all seven configurations. Single seed, 70/15/15 split. Highest value in each row in bold. The XGBoost column is retained as originally recorded but did not reproduce; see Section 7.7.

Class	RF	XGB	CNN	BiLSTM	Std Tf	IAA v1	IAA α=0
BENIGN	0.998	**1.000**	0.993	0.989	0.980	0.981	0.973
Bot	0.329	**0.790**	0.511	0.607	0.610	0.329	0.558
DDoS	**1.000**	**1.000**	0.999	0.989	0.986	0.978	0.975
DoS GoldenEye	0.995	**0.996**	0.988	0.972	0.886	0.839	0.842
DoS Hulk	0.996	**0.999**	0.989	0.976	0.923	0.942	0.858
DoS Slowhttptest	**0.992**	0.990	0.948	0.933	0.925	0.899	0.881
DoS slowloris	0.993	**0.994**	0.991	0.982	0.779	0.844	0.955
FTP-Patator	**1.000**	0.999	0.998	0.975	0.526	0.819	0.737
Heartbleed	**1.000**	0.667	0.800	**1.000**	0.500	0.133	0.500
Infiltration	**0.750**	**0.750**	0.667	0.261	0.028	0.045	0.038
PortScan	**0.997**	**0.997**	0.926	0.921	0.906	0.908	0.909
SSH-Patator	0.999	**1.000**	0.951	0.885	0.777	0.759	0.846
WA Brute Force	0.764	**0.820**	0.508	0.254	0.280	0.169	0.261
WA SQL Inj	0.000	**0.800**	0.051	0.010	0.009	0.011	0.007
WA XSS	**0.522**	0.203	**0.522**	0.129	0.165	0.144	0.153
Macro F1	**0.822**	**0.867**	**0.789**	**0.726**	**0.619**	**0.587**	**0.633**
Accuracy	**99.59%**	**99.89%**	**98.75%**	**98.10%**	**96.60%**	**96.82%**	**95.61%**

As originally recorded, XGBoost has the highest F1 on nine of the fifteen classes and the highest macro F1 overall, and no Transformer configuration leads on any class except where a tree model scores zero. The replicated comparison in Section 7.4 supersedes this table and gives a different ordering.

The XGBoost column specifically did not reproduce when the classical baselines were re-run. The three cells that drive its advantage, Heartbleed at 0.667, Infiltration at 0.750 and SQL Injection at 0.800, all fall to near zero on replication. Section 7.7 documents this in full. The Random Forest column reproduces and should be treated as the reliable classical baseline. Readers should not rely on the XGBoost figures in this table.

7.2 The XSS result does not survive precision

An earlier draft of this report headlined XSS recall rising from 0.58 to 0.95 as the principal evidence that the IAA mechanism worked. Including precision reverses that reading.

Table 4. Full precision, recall and F1 for the standard Transformer and IAA v1. Single seed, 70/15/15 split.

Class	Std P	Std R	Std F1	IAA P	IAA R	IAA F1	Δ F1
BENIGN	0.991	0.969	0.980	0.991	0.971	0.981	**+0.001**
Bot	0.570	0.655	0.610	0.217	0.679	0.329	**-0.281**
DDoS	0.996	0.976	0.986	0.978	0.978	0.978	**-0.008**
DoS GoldenEye	0.851	0.925	0.886	0.796	0.888	0.839	**-0.047**
DoS Hulk	0.897	0.949	0.923	0.901	0.986	0.942	**+0.019**
DoS Slowhttptest	0.879	0.976	0.925	0.830	0.981	0.899	**-0.026**
DoS slowloris	0.689	0.897	0.779	0.804	0.888	0.844	**+0.065**
FTP-Patator	0.574	0.485	0.526	0.696	0.996	0.819	**+0.293**
Heartbleed	0.333	1.000	0.500	0.071	1.000	0.133	**-0.367**

Infiltration	0.014	0.400	0.028	0.024	0.400	0.045	**+0.017**
PortScan	0.839	0.986	0.906	0.892	0.924	0.908	**+0.002**
SSH-Patator	0.922	0.671	0.777	0.798	0.725	0.759	**-0.018**
WA Brute Force	0.184	0.582	0.280	0.629	0.098	0.169	**-0.111**
WA SQL Inj	0.004	0.333	0.009	0.006	0.333	0.011	**+0.002**
WA XSS	0.096	0.577	0.165	0.078	0.949	0.144	**-0.021**
Macro	**0.589**	**0.781**	**0.619**	**0.581**	**0.786**	**0.587**	**-0.032**

XSS recall does rise, from 0.577 to 0.949. Precision, however, falls from 0.096 to 0.078, and F1 falls with it from 0.165 to 0.144. Expressed as counts, the standard Transformer detects 56 of the 97 XSS flows while predicting XSS on approximately 583 flows, yielding around 527 false positives. The IAA model detects 92 while predicting XSS on approximately 1,183 flows, yielding around 1,091 false positives. The false alarm count approximately doubled in exchange for 36 additional detections.

The gain is also not free. Web Attack Brute Force, the class most confusable with XSS, sees recall fall from 0.582 to 0.098 and F1 from 0.280 to 0.169 in the same run. Macro F1 across all fifteen classes falls from 0.619 to 0.587. The aggregate moved in the wrong direction while a single per-class recall figure moved in the right one.

7.3 The architectural advantage was an implementation artefact

The custom implementation outperformed PyTorch's nn.TransformerEncoderLayer by 0.122 macro F1, consistently across three seeds, with per-seed gaps of +0.130, +0.112 and +0.124. To establish the cause, the custom attention module was compared line by line against the PyTorch source. Two differences emerged: nn.MultiheadAttention applies dropout to the softmax output, which the custom implementation did not, and it initialises projection weights with xavier_uniform_ rather than the nn.Linear default.

Table 5. Isolating the source of the architectural difference. Three seeds each, 70/15/15 split.

Condition	Macro F1	Gap vs standard
Standard Transformer (nn.TransformerEncoderLayer)	0.506 ± 0.027	n/a
Custom architecture, alpha = 0	**0.628 ± 0.034**	**+0.122**
+ attention dropout 0.3 restored	0.534 ± 0.026	+0.028
+ xavier initialisation restored	0.606 ± 0.015	+0.100

Restoring attention dropout reduced macro F1 from 0.628 to 0.534, accounting for 77% of the gap. Xavier initialisation accounted for a further 18%. The two were tested independently, so their contributions are not strictly additive. The advantage of the custom implementation was an unintended absence of regularisation rather than an architectural contribution.

7.4 Model family comparison

All six model families were re-run at three seeds on the 80/20 split, so the comparison below rests on replicated results rather than single runs.

Table 6. Macro F1 by model family, three seeds each, 80/20 split.

Model	Macro F1	Standard deviation	Train time
Random Forest + SMOTE	**0.8985**	**0.0037**	**96 s**
1D-CNN	0.8216	0.0064	160 s
Random Forest	0.8194	0.0028	84 s
BiLSTM	0.7141	0.0066	566 s
Transformer	0.6315	not replicated	2,250 s
XGBoost	0.4856	0.0000	84 s

Random Forest with SMOTE is the strongest model in the study, exceeding the best Transformer by 0.267 macro F1 while training in 96 seconds against 2,250, a factor of roughly 23.

The 1D-CNN and Random Forest are indistinguishable. Their means differ by 0.0022 against a combined standard deviation of approximately 0.0069. The claim that tree ensembles outperform deep learning generally is therefore not supported. What the data supports is narrower: the Transformer specifically underperforms, trailing the CNN by 0.19 while requiring fourteen times the training time. A convolutional architecture on the same features, with the same preprocessing, matches the tree baseline.

XGBoost is the weakest model tested. Its result is discussed in Section 7.7, since the figure reported in an earlier draft did not reproduce.

The gap is not explained by imbalance handling, which was tested directly. Random Forest was run with class weighting and with SMOTE, three seeds each. McNemar's test on paired predictions gives chi-squared 1,052 in favour of the SMOTE configuration, significant below p equal to 0.001. Both configurations exceed the Transformer by a wide margin.

Training time was not instrumented for the multiclass runs, so no cost comparison is available for this table. Both were measured for the binary task in Section 9.4, where XGBoost trains in 7.6 seconds against 3,495 seconds for the Transformer.

This is broadly consistent with work reporting that tree ensembles remain competitive with deep architectures on tabular data, though the CNN result qualifies it. Trees split on raw feature values and are unaffected by differences in scale or distribution between columns. A Transformer must first project 71 unordered features of differing units into a shared embedding space before attention can operate, and in this setting that projection appears to discard more than the attention mechanism recovers. A convolutional model, which applies local filters across adjacent feature positions without requiring a shared embedding, does not pay that cost and matches the tree baseline.

The figures in Table 3 are single-seed and predate the replication work. Table 6 supersedes them. Both tree models have since been run with and without SMOTE at three seeds, so model family and resampling strategy are no longer confounded.

7.5 Rare-class metrics are unstable across seeds

Table 7. Identical configurations differing only in random seed.

Condition	Seed 42	Seed 123	Seed 456	Spread
Standard Transformer, XSS recall	0.258	0.031	0.526	**0.50**
IAA (alpha learned), XSS recall	0.887	0.299	0.887	**0.59**
IAA (alpha = 0), Bot F1	0.558	0.600	0.574	**0.04**

Part of this spread is attributable to the estimator rather than the model. XSS has 97 validation samples, so recall moves in increments of approximately 0.01; SQL Injection has three, so it moves in increments of 0.33. Classes with adequate support remain stable, as the Bot row demonstrates. The practical consequence is that single-seed results on the rarest classes are uninformative, and differences between conditions are meaningful only when they exceed approximately ±0.03 macro F1.

7.6 SMOTE and loss-level methods behave differently

Table 8. Effect of SMOTE on per-class F1, seed 42, 70/15/15 split.

Class	Support	F1 without SMOTE	F1 with SMOTE	Change
Web Attack - Brute Force	225	0.044	0.261	**+0.217**
Web Attack - XSS	97	0.000	0.153	**+0.153**
DoS slowloris	867	0.836	0.955	**+0.119**
SSH-Patator	882	0.755	0.846	**+0.091**
Bot	293	0.468	0.558	**+0.089**
FTP-Patator	1187	0.976	0.737	**-0.239**

SMOTE improves F1 rather than recall alone, and precision rises for several classes: DoS slowloris from 0.816 to 0.934, SSH-Patator from 0.676 to 0.889. Three classes are never predicted at all without it.

The loss-level methods behave differently. LDAM-DRW at seed 42 raised DoS slowloris recall to 0.837 but reduced precision to 0.222, taking F1 from 0.955 to 0.351. Bot F1 fell from 0.558 to 0.230 and macro F1 from 0.628 to 0.414. Class-weighted cross entropy showed the same pattern: an inverse-frequency configuration collapsed training entirely to macro F1 0.005, and a log-scaled variant reached 0.441.

The difference is mechanical. SMOTE supplies additional training examples within the minority region, allowing the model to learn a better boundary, so both precision and recall improve. LDAM and class weighting operate on the same examples and displace the decision boundary outward, which necessarily admits additional false positives. FTP-Patator is the sole exception, regressing under SMOTE on a class with 1,187 validation samples. This constitutes a genuine regression rather than a small-sample artefact.

7.7 A reproducibility failure in the classical baselines

While running the outstanding experiments described in Section 9, the XGBoost result reported above was found not to reproduce. This section documents the discrepancy, since the affected figure was the headline result of an earlier draft.

Table 9. XGBoost macro F1 under three conditions, identical hyperparameters and seed.

Run	Split	Software	Macro F1
Original notebook	70/15/15	local environment, version not recorded	**0.8670**
Verification	70/15/15	Narval, XGBoost 3.2.0	**0.6925**
Follow-up experiments	80/20	Narval, XGBoost 3.2.0	**0.4856**

The verification run used the same split files, the same hyperparameters and the same random seed as the original. It returns 0.6925 rather than 0.8670. The only variable that differs is the software environment.

Table 10. Where the discrepancy is concentrated.

Class	Test samples	Notebook F1	Verification F1	Difference
Heartbleed	**2**	0.6667	0.0000	**-0.6667**
Infiltration	**5**	0.7500	0.0000	**-0.7500**
Web Attack - SQL Injection	**3**	0.8000	0.1111	**-0.6889**
Web Attack - Brute Force	225	0.8195	0.6680	-0.1515
Bot	293	0.7903	0.7113	-0.0790
All other classes	various	above 0.99	above 0.90	under 0.09 each

The three classes with fewer than six test samples account for 0.140 of the 0.174 total difference. In the notebook run XGBoost correctly classified one of two Heartbleed flows, three of five Infiltration flows and two of three SQL Injection flows. In the verification run it classified almost none of them correctly. No property of the model changed.

This is the phenomenon documented in Section 7.5 applied to the project's own headline figure. The report argues that rare-class metrics on this dataset are unstable to the point of being uninformative below roughly twenty samples. The 0.867 was itself an instance of that instability, and it was not identified until the classical baselines were re-run at multiple seeds.

Table 11. Reproducibility of the two tree models.

Model	Notebook	Narval, three seeds	Reproduces
Random Forest	0.822	0.8194 ± 0.0028	**yes**
XGBoost	0.867	0.4856 ± 0.0000	**no**

Random Forest reproduces to within 0.003 despite a different split and a different software environment. XGBoost does not. Both models are deterministic given a seed, and XGBoost shows a standard deviation of exactly zero across seeds, so the instability is not seed variance. It arises from the interaction between a version change and classes small enough that a single prediction moves the class F1 by half or more.

Two consequences follow. First, Random Forest rather than XGBoost is the strongest classical model in this study, and the comparison in Section 7.4 has been rewritten accordingly. Second, any macro-averaged metric computed over a class set that includes classes with single-digit sample counts is fragile to changes that have nothing to do with the model. Reporting such an average without the per-class breakdown alongside it is not sufficient, a point the earlier draft of this report demonstrated at its own expense.

8. Ablation

The ablation was run with alpha hard-fixed at zero using the identical code path, the same three seeds and requires_grad disabled, so the only difference between the two conditions is the contribution of the frequency bias term.

Table 12. Ablation of the frequency bias term. Macro F1, three seeds, 70/15/15 split.

Seed	Alpha learned	Alpha = 0	Difference
42	0.574	0.633	**-0.059**
123	0.561	0.592	**-0.032**
456	0.638	0.659	**-0.021**

Alpha equal to zero matched or outperformed the freely learned condition on every seed. Two of the three differences fall within the seed noise of approximately ±0.03 established in Section 7.5 and are therefore not individually conclusive. The result is nonetheless persuasive because all three differences share the same sign. Under the null hypothesis of noise, at least one seed would be expected to favour the learned condition. The class frequency bias term contributes nothing to minority-class performance, and the central hypothesis of this project does not hold.

The outcome was foreshadowed during training. Across versions the learned alpha converged to values near zero or negative: -0.0684 in v5, -0.0048 in v5b and -0.0637 in v7. The optimiser was reducing the influence of the term rather than exploiting it.

8.1 Why the mechanism does not transfer to attention

The underlying difficulty is that class rarity is a property of the label distribution, whereas attention operates over features within a single sample. No principled mapping exists from a class-level frequency signal to a per-feature one, and at inference the true class is in any case unavailable. The original proposal specified a multi-pass design where first-pass class probabilities would weight the bias, removing the label dependence. That design has since been implemented and is reported in Section 9.5. It behaves as the other formulations did, changing at most 0.031% of predictions and leaving macro F1 unchanged.

9. Follow-up Experiments

Following the review meeting of 11 August with Dr. Passi and Dr. Rahman, four further experiments were carried out. All use an 80/20 stratified train/test split of the cleaned 71-feature data, without chronological ordering, replacing the earlier 70/15/15 split. The scaler is fitted on the training portion only and SMOTE, where applied, is confined to training data.

The model throughout is the custom Transformer with the frequency bias fixed at zero, since the ablation showed that term contributes nothing. Attention dropout remains absent and the projection layers retain the default nn.Linear initialisation, deliberately, so these results stay comparable with those above; the effect of each is quantified in Table 5.

On the output activation: nn.CrossEntropyLoss applies log-softmax internally, so the output layer returns raw logits. Adding an explicit softmax before the loss would apply the operation twice and flatten the gradients. Softmax is present in the model, inside the loss function rather than as a separate layer.

9.1 Experiment 1: Leaky ReLU in the hidden layers

The feed-forward block in each encoder layer previously used ReLU. This experiment substitutes Leaky ReLU with negative slope 0.01, which passes a small gradient for negative inputs rather than zero. A matched ReLU run on the same 80/20 split was added as a control, since the earlier ReLU figures were produced under the 70/15/15 split and are not directly comparable.

Table 13. ReLU versus Leaky ReLU, fifteen-class task, 80/20 split, seed 42.

Class	F1 (ReLU)	F1 (Leaky ReLU)	Change
BENIGN	0.9727	0.9645	**-0.0082**
Bot	0.4776	0.4356	**-0.0420**
DDoS	0.9505	0.9462	**-0.0043**
DoS GoldenEye	0.8482	0.8135	**-0.0347**
DoS Hulk	0.8562	0.7981	**-0.0581**

DoS Slowhttptest	0.9195	0.8883	**-0.0312**
DoS slowloris	0.9477	0.6542	**-0.2935**
FTP-Patator	0.9573	0.7654	**-0.1919**
Heartbleed	0.4000	0.3636	**-0.0364**
Infiltration	0.0825	0.0275	**-0.0550**
PortScan	0.9156	0.8489	**-0.0667**
SSH-Patator	0.8835	0.6619	**-0.2216**
WA Brute Force	0.1284	0.1460	**+0.0176**
WA SQL Inj	0.0131	0.0000	**-0.0131**
WA XSS	0.1191	0.1070	**-0.0121**
Macro F1	**0.6315**	**0.5614**	**-0.0701**
Accuracy	**95.52%**	**94.08%**	**-1.44%**

Leaky ReLU performs worse, not better. Macro F1 falls from 0.6315 to 0.5614, and the substitution is worse on fourteen of the fifteen classes. The gap of 0.0701 exceeds the seed-to-seed variation of approximately ±0.03 established in Section 7.5, so it is unlikely to be noise, though both conditions are single-seed and confirmation would require three runs each.

Two classes account for most of the loss. DoS slowloris falls from 0.9477 to 0.6542 and SSH-Patator from 0.8835 to 0.6619. Both are moderately rare classes with adequate support, so these are real degradations rather than small-sample artefacts. Only Web Attack Brute Force improves, by 0.0176, which is within noise.

The motivation for Leaky ReLU is to avoid units that stop updating because their gradient is zero for all inputs. That failure mode does not appear to be limiting this model, and the small negative slope evidently costs more than it recovers here.

9.2 Experiment 2: Binary classification

All fourteen attack classes are collapsed into a single ATTACK class and compared against BENIGN. This reframes the task as detecting malicious traffic rather than identifying which attack it is, and substantially reduces the imbalance: attacks constitute roughly a fifth of all flows rather than individual classes constituting fractions of a percent.

Table 14. Binary classification, Transformer, 80/20 split.

Class	Precision	Recall	F1	Support
BENIGN	0.9992	0.9435	0.9705	454,250
ATTACK	0.8121	0.9968	0.8950	111,311
Macro avg	**0.9056**	**0.9701**	**0.9328**	**565,561**
Accuracy			**95.40%**	

Confusion matrix, binary task.

	Predicted BENIGN	Predicted ATTACK
Actual BENIGN	428,578	25,672
Actual ATTACK	358	110,953

Attack recall is 0.9968: only 358 of 111,311 attack flows are missed. In security terms that is the metric that matters most, since a missed attack is a breach. Precision on ATTACK is 0.8121, however, meaning 25,672 benign flows are flagged as attacks. That is 5.65% of all legitimate traffic.

Whether this trade-off is acceptable depends on the deployment context. For an alerting system subject to manual analyst review, flagging one in eighteen benign flows would generate an unmanageable queue. The comparison in Section 9.4 demonstrates that this is not an inherent property of the task.

9.3 Experiment 3: Multiclass with SQL Injection removed

Web Attack SQL Injection has 21 samples in the full dataset, of which four fall in the test partition under an 80/20 split. At that scale a single prediction changes recall by 0.25, so no reliable estimate is possible. This experiment removes the class entirely and retrains on the remaining fourteen.

Table 15. Fourteen-class results, ReLU, 80/20 split, seed 42.

Class	Precision	Recall	F1	Support
BENIGN	0.9632	0.9829	0.9730	454,250
Bot	0.4906	0.6010	0.5402	391
DDoS	0.9982	0.9035	0.9485	25,605
DoS GoldenEye	0.7395	0.9456	0.8299	2,059
DoS Hulk	0.9732	0.7223	0.8292	46,025
DoS Slowhttptest	0.7553	0.9764	0.8517	1,100
DoS slowloris	0.9048	0.9508	0.9272	1,159
FTP-Patator	0.9921	0.9546	0.9730	1,587
Heartbleed	0.1053	1.0000	0.1905	2
Infiltration	0.0625	0.5714	0.1127	7
PortScan	0.8877	0.9528	0.9191	31,761
SSH-Patator	0.8344	0.9780	0.9005	1,180
Web Attack - Brute Force	0.1540	0.6246	0.2470	301
Web Attack - XSS	0.0905	0.4538	0.1509	130
Macro avg	**0.6394**	**0.8298**	**0.6710**	**565,557**

Macro F1 is 0.6710 across fourteen classes. Comparison against the fifteen-class result requires care, because removing a class scoring 0.0131 raises a fourteen-class average mechanically. Recomputing the ReLU control over the same fourteen classes gives 0.6756, against 0.6710 here.

The difference of -0.0047 is well inside noise. Removing SQL Injection therefore had no measurable effect on the remaining classes: the apparent improvement in the macro average is entirely arithmetic, produced by dropping a near-zero term from the mean rather than by any change in how the model handles the classes that remain.

This is a useful negative result in its own right. It indicates the extreme minority class was not interfering with the others, only making the aggregate metric harder to interpret.

9.4 Experiment 4: Model comparison on the binary task

Random Forest, Gaussian Naive Bayes and XGBoost were trained on the same binary task for comparison against the Transformer from Experiment 2. The tree models use raw unscaled features; Naive Bayes uses scaled features, as it assumes Gaussian inputs. Random Forest uses class_weight balanced; XGBoost and Naive Bayes receive no imbalance correction in this table. Both tree models were subsequently re-run with SMOTE at three seeds; those results appear in Section 7.4.

Table 16. Binary classification across four model families, 80/20 split.

Model	Accuracy	Prec (ATK)	Rec (ATK)	F1 (ATK)	Macro F1	Trai
XGBoost	**99.93%**	**0.9974**	**0.9988**	**0.9981**	**0.9988**	**7.6**
Random Forest	99.89%	0.9955	0.9991	0.9973	0.9983	75.5
Transformer	95.40%	0.8121	0.9968	0.8950	0.9328	3,495
Gaussian Naive Bayes	41.15%	0.2495	0.9908	0.3986	0.4113	1.5

XGBoost trains in 7.6 seconds and reaches macro F1 0.9988. The Transformer trains in 3,495 seconds, roughly 460 times longer, and reaches 0.9328. Random Forest sits between them on cost and close to XGBoost on performance.

The error counts make the gap concrete. The Transformer produces 25,672 false positives and 358 false negatives. XGBoost produces 289 false positives and 130 false negatives. Expressed as rates, the Transformer flags 5.65% of benign traffic as malicious against XGBoost's 0.064%, a difference approaching two orders of magnitude. In an operational setting this separates a deployable system from one that would be disabled.

Gaussian Naive Bayes performs poorly at 41.15% accuracy, below what predicting the majority class would achieve. Its assumptions of conditional feature independence and Gaussian marginals are both violated by network flow features, which are heavily correlated and strongly skewed. It is retained as a negative control, establishing that the binary task is not trivially separable for an arbitrary classifier.

The binary formulation simplifies the problem considerably relative to the fifteen-class task, but does not narrow the gap between model families. The gap in fact widens in relative terms. On the multiclass task, using the same 80/20 split, the Transformer reached 0.6315 against 0.8985 for the best tree configuration. On the binary task it reaches 0.9328 against 0.9988. The proportion of remaining error that the Transformer fails to close is larger in the binary case. The tree models exploit the simpler decision boundary more effectively than the Transformer.

9.5 Multi-pass inference

Section 6.4 of the original project proposal specified that, since the class frequency is unavailable from labels at inference, the model should instead use the learned scaling parameter together with the class probabilities predicted by a first pass. This design was never implemented during the main body of the work. The versions that were built read the bias directly from the true label, which is the leakage identified in the review. This experiment implements the design as originally specified.

The model runs twice per sample. The first pass produces logits and a probability distribution over the fifteen classes. The bias each class receives in the second pass is scaled by the probability the first pass assigned to it, so the term becomes alpha multiplied by p_c multiplied by log of one over f_c. No label is read at any point.

Table 17. Multi-pass inference, three seeds. Both passes evaluated separately.

Seed	Learned alpha	Predictions changed	Macro F1 pass 1	Macro F1 pass 2
42	0.0022	41 (0.007%)	0.6335	0.6335
123	-0.0062	173 (0.031%)	0.5766	0.5766
456	0.0027	19 (0.003%)	0.6453	0.6450

The second pass changes between 19 and 173 predictions out of 565,561, at most 0.031% of the test set. Macro F1 is identical to four decimal places on two of the three seeds and lower by 0.0003 on the third. The learned alpha converges to values between -0.006 and 0.003 in every case, so the model reduces the term to approximately zero of its own accord.

This closes the last untested formulation of the original hypothesis. The multi-pass design does not leak labels, which was its purpose, and it also does not do anything. Taken together with the ablation in Section 8, the frequency bias term has now been tested at the attention level where it was mathematically inert, at the output level where it was well defined but contributed nothing, and in the multi-pass form specified in the proposal where it again contributes nothing.

9.6 Additional ablations

Two preprocessing decisions were tested directly. Retaining all seven sparse TCP flag columns gave macro F1 0.545 against 0.633 with them dropped, indicating the removal cost nothing on this data. Removing Destination Port as a feature moved the custom model from 0.628 to 0.559 and the standard Transformer from 0.506 to 0.539; both changed modestly, and the feature is retained with the caveat noted in Section 12.

9.7 Day-based split and duplicate statistics

Every result reported so far uses a random stratified split. CICIDS-2017 was captured over five working days, so a random split places flows from the same attack session on both sides. This experiment measures how much that matters.

Duplicate overlap under the random split

Each row was hashed and the partitions compared. Within the training set, 229,377 rows are exact duplicates of another training row, 10.14% of the partition. More importantly, 75,950 test rows are byte-identical to a row present in training, which is 13.43% of the test set.

Table 18. Test flows that appear identically in the training partition.

Class	Test flows	Also in training	Percent
PortScan	31,761	18,683	**58.8%**
SSH-Patator	1,180	555	**47.0%**
FTP-Patator	1,587	468	**29.5%**
DoS Hulk	46,025	11,756	**25.5%**
BENIGN	454,250	44,289	9.7%
DoS slowloris	1,159	112	9.7%
DoS Slowhttptest	1,100	71	6.5%
Web Attack - Brute Force	301	7	2.3%
All remaining classes	under 26,000 each	6 or fewer	under 0.5%

For four classes the model can score well by recognising flows it has already memorised rather than by generalising. PortScan, where 58.8% of test flows appear identically in training, reaches F1 above 0.90 for every model in this study. That figure cannot be read as evidence of learned detection.

Day-based split

The eight capture files were split by day, training on five and testing on three. The multiclass task cannot be evaluated this way: several classes appear in a single capture file only, so any day-based partition removes them entirely from one side. Heartbleed appears only in the Wednesday capture and PortScan only in the Friday afternoon capture. The evaluation is therefore run on the binary attack-versus-benign task, where every day contains both classes.

Duplicate overlap falls from 13.43% under the random split to 4.92% under the day-based one.

Table 19. Binary task under both split strategies.

Model	Split	Macro F1	Attack recall	Attacks detected
Random Forest	random	0.9983	0.9991	111,215 of 111,311
Random Forest	**day-based**	**0.4509**	**0.0076**	**2,043 of 267,735**
XGBoost	random	0.9988	0.9988	111,181 of 111,311
XGBoost	**day-based**	**0.5275**	**0.0871**	**23,330 of 267,735**

Performance collapses. Random Forest detects 0.76% of attacks in the held-out days, having detected 99.91% under the random split. XGBoost detects 8.71%. Both retain high precision, above 0.99, so the models are not producing false alarms. They are simply failing to identify almost anything as an attack.

The cause is not solely duplicate overlap. Because attack families are unevenly distributed across capture days, the test partition contains families absent from training. The training days hold DDoS, PortScan, Bot and Infiltration; the test days hold the web attacks, the Patator family, the DoS variants and Heartbleed. The models are therefore being asked to detect attack types they have never seen, in addition to losing the memorised flows. This split cannot separate the two effects.

That limitation is itself a finding about the dataset. CICIDS-2017 cannot support a clean temporal evaluation, because holding out days necessarily also holds out attack families. Any published result on this dataset that uses a random split, which is the overwhelming majority, is measuring something closer to interpolation within known attack sessions than detection of unseen traffic. The 99.9% binary accuracy reported throughout the literature, and in Section 9.4 of this report, should be read with that in mind.

10. Conclusion

The central hypothesis of this project was that incorporating class frequency into a Transformer's attention mechanism would improve detection of rare attack classes. It does not. An ablation with alpha fixed at zero, run over three seeds on an identical code path, matched or outperformed the learned model on every seed. The frequency bias term contributes nothing.

An apparent architectural advantage of 0.122 macro F1 over the standard Transformer was traced to two unintended implementation differences rather than to design. The absence of dropout on attention weights alone accounted for 77% of the gap.

The headline result of an earlier draft does not survive scrutiny. XSS recall rising from 0.58 to 0.95 was reported as evidence the mechanism worked; including precision shows F1 falling from 0.165 to 0.144, with the gain absorbed from Web Attack Brute Force, whose F1 fell from 0.280 to 0.169. Macro F1 across all classes fell by 0.032. This is a direct illustration of why recall alone is uninterpretable on rare classes.

Loss-level alternatives were also evaluated. Class-weighted cross entropy and LDAM-DRW both raised rare-class recall while reducing precision by a greater margin, leaving F1 below the unmodified baseline. This follows from their mechanism: they shift the decision boundary outward without supplying additional information about the minority region. SMOTE behaves differently, improving both precision and recall for several classes, and remains the only intervention tested that improved F1.

Two results emerged that the project did not set out to find. The first concerns model families. Random Forest with SMOTE reaches macro F1 0.8985 across three seeds against 0.6315 for the best Transformer, and trains in 96 seconds against 2,250. A 1D-CNN reaches 0.8216 and is statistically indistinguishable from Random Forest, so the deficit is specific to the Transformer rather than general to deep learning. For this feature representation, the choice of architecture matters more than the imbalance-handling strategy applied within it.

An earlier draft attributed this result to XGBoost at macro F1 0.867. That figure was found not to reproduce during the follow-up work. Re-running the identical configuration returns 0.6925 on the same split and 0.4856 on the new one, with the difference concentrated almost entirely in three classes holding two, five and three test samples. Random Forest, by contrast, reproduces to within 0.003. The correction is documented in Section 7.7. The correction does not alter the finding that the Transformer trails the other model families, but it changes which classical model leads, and it is an instance of the project's own finding about rare-class instability appearing in its own headline number.

The follow-up experiments requested at the review meeting of 11 August reinforce rather than qualify these findings. Substituting Leaky ReLU for ReLU degraded macro F1 from 0.6315 to 0.5614, worse on fourteen of fifteen classes. Removing SQL Injection changed nothing measurable in the remaining fourteen: the apparent improvement in the macro average is arithmetic, not behavioural. Reframing the task as binary attack detection raised the Transformer to macro F1 0.9328, but XGBoost reached 0.9988 on the same task in 7.6 seconds against 3,495 seconds of GPU time, and produced 289 false positives where the Transformer produced 25,672.

The second concerns the dataset itself. Replacing the random split with a day-based one collapses binary performance from macro F1 0.998 to between 0.45 and 0.53, with attack recall falling from above 0.999 to below 0.09. Random Forest detects 2,043 of 267,735 attacks in held-out days. Under the random split, 13.43% of test flows are byte-identical to a training flow, rising to 58.8% for PortScan. Part of the collapse reflects attack families that appear on only one capture day and are therefore absent from training under any temporal split, so the two effects cannot be separated here. Either way, results obtained from a random split on this dataset do not estimate deployment performance.

For the rarest classes the question is moot regardless. Heartbleed has eleven samples in the full dataset and SQL Injection twenty-one. With two to four samples in any test partition, a single prediction changes recall by a third, and no method can be reliably evaluated at that scale. Reported metrics for these classes should be read as noise irrespective of the model producing them.

11. Contributions

Stated plainly, and none of them are what this project set out to find:

A negative result on attention-level class frequency bias for tabular intrusion detection, established by ablation over three seeds on an identical code path.

A traced explanation of an apparent architectural gain, showing it to be an artefact of absent regularisation rather than design. The general lesson is that architectural comparisons against library implementations require a line-by-line source diff before any difference is attributed to design.

A demonstration that recall reported without precision produced a false positive finding in this work, and that correcting it reversed the headline conclusion.

Evidence that rare-class metrics on CICIDS-2017 are unstable across random seeds to a degree that renders single-seed reporting uninformative: XSS recall varied from 0.031 to 0.526 across three seeds for an identical configuration.

A replicated model-family comparison at three seeds showing Random Forest with SMOTE strongest at 0.8985, a 1D-CNN statistically indistinguishable from Random Forest, and the Transformer trailing both by roughly 0.19 while requiring an order of magnitude more compute. The deficit is specific to the Transformer rather than general to deep learning.

Evidence that performance on CICIDS-2017 is substantially an artefact of partitioning. Thirteen percent of test flows under a random split are byte-identical to a training flow, and moving to a day-based split collapses binary macro F1 from 0.998 to between 0.45 and 0.53.

A completed test of every formulation of the original hypothesis, including the multi-pass inference design specified in the proposal, which changes at most 0.031% of predictions and leaves macro F1 unchanged.

A documented reproducibility failure in the project's own earlier headline result, traced to three classes with fewer than six test samples, together with evidence that Random Forest reproduces across environments where XGBoost does not.

A demonstration that the gap between model families widens rather than narrows when the task is simplified: on the binary formulation the Transformer produces 25,672 false positives against XGBoost's 289, at roughly 460 times the training cost.

12. Limitations

SMOTE at the ratios used here is not credible for the extreme minority classes. Heartbleed goes from eleven real samples to ten thousand synthetic ones, a roughly 900-fold interpolation within the convex hull of eleven points in 71 dimensions. Those samples are near-duplicates rather than new information, and any Heartbleed figure reported here may reflect memorised synthetic patterns rather than learned attack behaviour.

Rows containing NaN or infinite values were removed from all splits including the test set. In deployment such records still arrive, so the test distribution here is cleaner than production would be, which may inflate reported performance. Retaining them and imputing would be the correct treatment.

Destination Port was retained as a numeric feature. This permits the model to memorise port-to-attack mappings from the lab environment rather than learning generalisable traffic behaviour. The ablation in Section 9.5 shows modest movement, but the concern stands for external validity.

The main results use a random stratified split. A day-based split and duplicate statistics were subsequently measured and are reported in Section 9.7. Both indicate that results obtained under the random split overstate what the models would achieve on unseen traffic. The figures in Sections 7 and 9.1 to 9.4 should be read as upper bounds.

CICIDS-2017 contains systematic labelling and CICFlowMeter feature extraction errors documented by Engelen et al. (2021). This matters more here than in most work, since every claim concerns rare classes, where a handful of mislabels dominates.

The confound between model family and resampling strategy has since been removed. Both tree models were re-run with SMOTE and with class weighting, three seeds each, and all classical models are now reported with variance. The comparison in Section 7.4 rests on replicated results throughout.

13. Future Work

Collecting additional real samples for the extreme minority classes is the most critical next step. Below approximately twenty real samples, no architecture can be reliably evaluated irrespective of how well it learns during training.

A temporal evaluation that holds attack families constant across the split would separate the two causes of the collapse reported in Section 9.7. CICIDS-2017 cannot support this, since several attack families appear on a single capture day. A dataset with attacks distributed across days would be required.

Validation on a more modern secondary benchmark such as UNSW-NB15 or CSE-CIC-IDS2018 would establish whether these findings generalise, particularly the gradient boosting result and the rare-class variance. NSL-KDD, originally planned as the secondary dataset, derives from 1998 traffic and shares KDD99's pathologies.

Per-class threshold calibration rather than a fixed decision boundary, and seed-ensembling to average out training noise, are standard production techniques that were not explored here and would directly address the precision-recall trade-off documented in Section 7.2.

Adding SHAP explainability would allow visualisation of which flow features drive each prediction, making the model interpretable for security analysts who need to understand why an alert fired.

14. Note on Reproducibility

Two output file collisions occurred during the project. In both cases a script was produced from an existing one by editing only the line under test, which left the output path unchanged, so the second job to finish overwrote the first report file.

The first affected v5 and v5b. The second affected the alpha-learned and alpha-zero conditions at seed 42. Seeds 123 and 456 are unaffected, as are all other runs.

No results were lost. SLURM writes a separate log per job, so the overwritten figures were recovered from those logs and are the ones reported here. The practice of deriving one script from another by targeted edit should nonetheless include a change to the output path, and did not.

Appendix A. Experimental Record

This appendix records every configuration trained during the project, what each was intended to test, and what it showed. It is included because several findings in the main report depend on comparisons across configurations, and because the sequence in which the mechanism was corrected is itself part of the evidence.

All runs were carried out on the Narval HPC cluster under allocation def-kpassi, using A100 GPU nodes for the deep models and CPU nodes for the tree models. Fifty-nine training runs were completed in total, itemised in Table A9.

A.1 Development of the IAA mechanism

Table A1. Successive implementations of the frequency bias term. Macro F1 on the validation set.

Version	What changed	Macro F1	Outcome
v1	Bias inside attention, post-SMOTE frequencies, alpha initialised 1.0	0.5867	Inert, softmax cancelled
v2	Bias inside attention, original dataset frequencies	0.6143	Inert
v3	Added torch.abs constraint on alpha	0.5978	Inert
v4	abs constraint with post-SMOTE frequencies	0.5823	Inert
v5	Bias moved to output logits	0.5680	First valid implementation
v5b	v5 with torch.abs on alpha	0.6188	Valid, stable
v6	v5b with inverse-frequency weighted loss	0.0053	Training collapsed
v7	v5b with log-scaled weighted loss	0.4408	Unstable, poor precision

Versions one through four were mathematically inert for the reason set out in Section 4.1: a scalar bias broadcast across the key dimension is cancelled by softmax. This was not identified until the written review of 6 August by Dr. SK Md Mizanur Rahman. The differences in macro F1 between those four rows reflect seed and initialisation variance rather than any effect of the mechanism.

Version six failed because inverse-frequency weights gave BENIGN an effective weight of approximately zero, so the model stopped predicting the majority class entirely. Version seven used log-scaled weights, which trained stably but traded precision for recall across every minority class.

The v5 figure of 0.5680 was read from the SLURM job log rather than the saved report. The v5b script was produced from the v5 script by editing only the alpha line, so the output path was unchanged and v5b overwrote v5's report file. This is the same collision described in Section 14, which affected the seed 42 ablation. In both cases the job logs are written separately per job and preserve the original figures.

A.2 Ablation and mechanism isolation

Table A2. Runs isolating the contribution of individual components.

Configuration	Seeds	Purpose	Result
IAA, alpha learned	42, 123, 456	Baseline for the ablation	0.591 ± 0.041
IAA, alpha fixed at 0	42, 123, 456	**True ablation, identical code path**	**0.628 ± 0.034**
Standard Transformer	42, 123, 456	Library baseline	0.506 ± 0.027
+ attention dropout 0.3	42, 123, 456	Isolate the missing dropout	0.534 ± 0.026
+ xavier initialisation	42, 123, 456	Isolate the initialisation difference	0.606 ± 0.015
Attention dropout sweep	42	Characterise the dropout effect	0.0 to 0.3, see A3

Table A3. Attention dropout sweep, single seed.

Dropout	Macro F1	Bot F1
0.0	0.633	0.558
0.1	0.628	0.507
0.2	0.649	0.461
0.3	**0.544**	**0.221**

Values from 0.0 to 0.2 differ by 0.021 macro F1, which falls below the seed-to-seed variation of ±0.03 established in Section 7.5, and cannot be distinguished. Only 0.3 lies clearly outside that band. Bot F1 declines monotonically across the range while macro F1 does not, so no dose-response relationship can be claimed from a single seed.

A.3 Preprocessing ablations

Table A4. Runs testing individual preprocessing decisions.

Configuration	Macro F1	What it establishes
IAA baseline (alpha = 0, seed 42)	0.633	Reference
Without SMOTE	0.627	Three classes never predicted at all
Standard Transformer without SMOTE	0.407	SMOTE is load-bearing for both architectures
Without Destination Port	0.559	Modest dependence on the port feature
Standard Transformer without Destination Port	0.539	Same direction for the library model
TCP flag columns retained	0.545	Dropping the seven sparse flags cost nothing
LDAM-DRW loss	0.414	Recall up, precision down further

A.4 Cross validation

Five-fold stratified cross validation was run for both the IAA-Transformer and the standard Transformer. Both models saw identical folds, generated from the same StratifiedKFold object with the same random seed, and SMOTE was applied inside each training fold only. This was carried out in response to the observation that comparing a five-fold average against a single held-out split is not a valid comparison.

Table A5. Five-fold cross validation, paired protocol.

Metric	Standard Transformer	IAA-Transformer	Difference
Macro F1	0.447 ± 0.034	0.555 ± 0.025	**+0.108**
XSS recall	0.108	0.711	+0.603
XSS F1	0.039 ± 0.020	0.132 ± 0.025	+0.093
Bot F1	0.312 ± 0.149	0.382 ± 0.083	+0.070
SQL Injection recall	0.250	0.350	+0.100

Table A6. Paired per-fold macro F1.

Fold	Standard Transformer	IAA-Transformer	Difference
1	0.4457	0.5361	+0.0904
2	0.4147	0.5330	+0.1183
3	0.4770	0.5960	+0.1190
4	0.4129	0.5510	+0.1381
5	0.4851	0.5593	+0.0742
Mean	**0.4471**	**0.5551**	**+0.1080**

A paired t-test over the five folds gives a mean difference of +0.1080 with standard error 0.0114, t equal to 9.50 on four degrees of freedom. The critical value at four degrees of freedom is 4.604 for p below 0.01, so the difference is significant at that level. The 95% confidence interval is +0.0764 to +0.1396.

A Wilcoxon signed-rank test is also reported for completeness. All five differences are positive, which is the strongest result the test can produce at this sample size, but with five pairs the smallest attainable two-tailed p value is 0.0625. The test cannot reach conventional significance regardless of the data, so the t-test is the more informative of the two here. Both are underpowered with five folds and should be read as descriptive.

On the accuracy discrepancy: the cross validation runs average 94.69% for the IAA-Transformer against 96.82% reported in the single-split experiments. The two are not measuring the same thing. The cross validation combined the training and validation partitions and re-split them, so each fold trains on a different subset and evaluates on data the single-split model never saw. Whether that fully accounts for two percentage points has not been verified, and the discrepancy is noted rather than explained.

Under the paired protocol the IAA-Transformer is ahead on macro F1. This result is not contradicted by the ablation in Section 8: the ablation compares alpha learned against alpha fixed at zero within the same architecture, whereas this table compares that architecture against PyTorch's. Section 7.3 establishes that the architectural difference is attributable to absent regularisation, so the k-fold gap should be read as measuring that rather than the frequency bias.

A.5 Baseline models

Table A7. Baseline configurations and training cost.

Model	Configuration	Epochs	Imbalance handling	Train time
Random Forest	100 trees, max_depth 20	n/a	class_weight balanced	96 s (CPU)
XGBoost	100 estimators, max_depth 8, lr 0.3	n/a	None	99.5 s (CPU)
1D-CNN	Conv1d 1→64→128, kernel 3, MaxPool 2, FC 256, dropout 0.3	10	SMOTE	not recorded
BiLSTM	input_size 1, hidden 128, 2 layers, bidirectional, dropout 0.3	10	SMOTE	not recorded
Transformer v2	d_model 128, nhead 4, 3 layers, ff 512, dropout 0.3	20	SMOTE	not recorded
IAA-Transformer	as Transformer v2, custom attention module	20	SMOTE	~37 min (A100)

An earlier Transformer at d_model 64 with two layers was discarded as too weak to serve as a baseline. Transformer v2 with the larger model dimension and longer training was used for all subsequent comparisons.

Table A8. Parameter counts and measured inference latency.

Model	Parameters	Inference (565,561 flows)	Per flow
Random Forest	100 trees, depth 20	0.67 s	0.0012 ms
XGBoost	100 trees, depth 8	0.19 s	0.0003 ms
Transformer, 15-class	597,007	10.6 s	0.0187 ms
Transformer, 14-class	596,878	10.4 s	0.0184 ms
Transformer, binary	595,330	10.4 s	0.0184 ms

Inference latency was measured on the follow-up experiments only, on the same 565,561 flow test set. The Transformer is roughly fifty times slower per flow than XGBoost at inference and fifteen times slower than Random Forest, on top of the training cost difference. For a system that has to keep pace with line-rate traffic this compounds the argument in Section 7.4.

Parameter counts are not directly comparable across families, since a tree ensemble has no equivalent quantity, but the tree configurations are given for reference.

Training times were not instrumented for the deep baselines and are marked accordingly rather than estimated. The Random Forest and XGBoost figures are printed by their notebooks. The IAA-Transformer figure comes from the follow-up experiments in Section 9, where timing was added to the training script. Times are wall clock on whichever device the model ran on and are not normalised across CPU and GPU.

A.6 Summary of runs

Table A9. Runs by category.

Category	Runs
IAA mechanism development (v1 to v7)	8

Multi-seed ablation and mechanism isolation	18
Attention dropout sweep	4
Preprocessing ablations	7
Five-fold cross validation, both models	10
Baseline models	6
Follow-up experiments (Section 9)	6
Total	**59**

Not all runs are reported individually in the main text. Those omitted either duplicate a reported configuration at a different seed, or belong to the inert versions v1 through v4 whose results carry no information about the mechanism under test.

A.7 Confusion matrices

Confusion matrices were produced for every follow-up experiment and saved alongside the classification reports. The fifteen-class matrix is too wide to render legibly at page width, so the principal confusions are given below, taken from the ReLU control run of Experiment 1. Entries are counts of test flows.

Table A10. Largest off-diagonal entries, fifteen-class ReLU run.

True class	Predicted as	Count	Reading
DoS Hulk	BENIGN	9,121	Largest single error. 20% of Hulk flows missed.
BENIGN	PortScan	3,230	False alarms on scanning behaviour
BENIGN	DoS Hulk	3,189	False alarms on volumetric patterns
DDoS	BENIGN	2,280	9% of DDoS flows missed
PortScan	BENIGN	2,186	7% of PortScan flows missed
BENIGN	WA Brute Force	2,036	Drives Brute Force precision to 0.073
BENIGN	WA XSS	968	Drives XSS precision to 0.066
WA Brute Force	WA XSS	114	38% of Brute Force flows
WA XSS	WA Brute Force	49	38% of XSS flows
WA SQL Inj	SSH-Patator	62	On a class with four test samples

The four shaded rows are the mechanism behind the finding in Section 7.2. Web Attack Brute Force and Web Attack XSS are confused with each other in both directions, and both are heavily over-predicted on benign traffic. Together the two classes have 431 test flows, while the model assigns more than three thousand flows to them. Any apparent gain on one is therefore likely to be movement between the two, or absorption from BENIGN, rather than genuine improvement.

The largest error overall is DoS Hulk misclassified as BENIGN, at 9,121 flows. This is a majority class failing rather than a rare-class one, and it is not addressed anywhere in this project.

Appendix B. Code Listings

The listings below are the parts of the implementation that the findings in this report depend on. The full codebase, including all training scripts and notebooks, is in the project repository.

B.1 The attention module

This is the custom scaled dot-product attention used in every Transformer configuration reported here. Two properties are relevant to Section 7.3. No dropout is applied to the softmax output, whereas PyTorch's nn.MultiheadAttention applies it by default. The four projection layers also use the default nn.Linear initialisation rather than xavier_uniform_.

```
class IAAAttention(nn.Module):
    def __init__(self, d_model, nhead):
        super().__init__()
        self.d_model = d_model
        self.nhead   = nhead
        self.d_head  = d_model // nhead
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x):
        b, s, _ = x.shape
        Q = self.W_q(x).view(b, s, self.nhead, self.d_head).transpose(1, 2)
        K = self.W_k(x).view(b, s, self.nhead, self.d_head).transpose(1, 2)
        V = self.W_v(x).view(b, s, self.nhead, self.d_head).transpose(1, 2)

        scores  = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_head)
        weights = F.softmax(scores, dim=-1)

        out = torch.matmul(weights, V)
        out = out.transpose(1, 2).contiguous().view(b, s, self.d_model)
        return self.W_o(out)
```

B.2 The bias placement that did not work

This is what versions one through four did. The bias tensor is one scalar per sample, reshaped to [batch, 1, 1, 1] and broadcast onto a score matrix of [batch, heads, 71, 71]. Every entry in a row receives the same value, and softmax normalises along that dimension, so the addition cancels and the attention weights are unchanged.

```
# v1 to v4. Mathematically inert.
scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_head)

bias   = self.alpha * class_bias.view(batch_size, 1, 1, 1)
scores = scores + bias        # cancelled by the softmax below

weights = F.softmax(scores, dim=-1)
```

For the bias to survive softmax it would need to vary along the key dimension, giving shape [batch, 1, 1, seq_len]. That raises a separate question the project could not answer, which is what a per-feature interpretation of a class-level frequency signal would actually mean. Section 8.1 covers this.

B.3 The bias placement that was valid

Version five moved the bias to the output logits. Each of the fifteen class scores receives a different value, so softmax cannot cancel it. freq_bias_all is a fixed fifteen-element tensor computed once from the training set frequencies, and it is added to every sample's logits regardless of that sample's true class. No label is read at inference.

```
# Computed once, from the training distribution.
freq_bias = torch.zeros(num_classes)
for i, label in enumerate(le.classes_):
    freq_bias[i] = math.log(1.0 / class_freq_original[label])
freq_bias = freq_bias.to(device)

# Inside IAATransformer.forward
x = x.mean(dim=1)
x = self.dropout(x)
x = self.fc(x)                                  # [batch, 15] raw logits
x = x + torch.abs(self.alpha) * freq_bias_all   # 15 different values
return x
```

B.4 The ablation

The ablation script is the training script with one line changed. Alpha is initialised at zero and requires_grad is disabled, so the bias term evaluates to zero throughout and the gradient never reaches it. Everything else, including the seed, the data loader generator and the optimiser state, is identical.

```
# Learned condition
self.alpha = nn.Parameter(torch.tensor(0.1))

# Ablation condition
self.alpha = nn.Parameter(torch.tensor(0.0), requires_grad=False)
```

Both scripts wrote to the same output path, so the report file for seed 42 was overwritten by whichever job finished second. The figures reported for that seed were read from the SLURM job logs, which are written separately per job. Section 14 records this.

B.5 Activation function, Experiment 1

The only change between the two conditions in Experiment 1 is which activation the feed-forward block uses. Note that the output layer returns raw logits in both cases. nn.CrossEntropyLoss applies log-softmax internally, so adding an explicit softmax before it would apply the operation twice and flatten the gradients.

```
def activation(kind):
    return nn.LeakyReLU(0.01) if kind == 'leakyrelu' else nn.ReLU()

self.feed forward = nn.Sequential(
    nn.Linear(d_model, ff),
    activation(act),
    nn.Dropout(dropout),
    nn.Linear(ff, d model))

# Output layer returns logits. Softmax is inside the loss.
criterion = nn.CrossEntropyLoss()
```

B.6 Data preparation

The 80/20 split used for the follow-up experiments, and the three label variants derived from it.

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test size=0.20, stratify=y, random state=SEED)

scaler = StandardScaler()
X_tr_s = scaler.fit_transform(X_tr)   # fitted on train only
X te s = scaler.transform(X te)

# Binary labels for Experiments 2 and 4
y_tr_bin = (y_tr != 'BENIGN').astype(int)
y te bin = (y te != 'BENIGN').astype(int)

# SQL Injection removed for Experiment 3
mask tr = y tr != 'Web Attack - Sql Injection'
mask_te = y_te != 'Web Attack - Sql Injection'

# SMOTE, training set only, minorities to 10,000
strategy = {i: 10000 for i in range(len(le.classes ))
            if np.sum(y_tr_enc == i) < 10000}
sm = SMOTE(sampling strategy=strategy, random state=SEED)
X_res, y_res = sm.fit_resample(X_tr_s, y_tr_enc)
```

B.7 Classical baselines

Configurations for the three classical models in Experiment 4. XGBoost receives no imbalance handling of any kind, which is the basis of the argument in Section 7.4. Naive Bayes uses the scaled features because it assumes Gaussian inputs.

```
RandomForestClassifier(n estimators=100, max depth=20,
                       class_weight='balanced',
                       random_state=SEED, n_jobs=-1)

GaussianNB()          # trained on scaled features
```

```
XGBClassifier(n estimators=100, max depth=8, learning rate=0.3,
              random state=SEED, n jobs=-1,
              eval_metric='logloss', verbosity=0)
              # no scale_pos_weight, no sample weights, no SMOTE
```

References

Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357.

Cui, Y., Jia, M., Lin, T. Y., Song, Y., and Belongie, S. (2019). Class-Balanced Loss Based on Effective Number of Samples. CVPR, 9268-9277.

Engelen, G., Rimmer, V., and Joosen, W. (2021). Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study. IEEE Security and Privacy Workshops, 7-12.

Gorishniy, Y., Rubachev, I., Khrulkov, V., and Babenko, A. (2021). Revisiting Deep Learning Models for Tabular Data. NeurIPS, 34, 18932-18943.

Huang, X., Khetan, A., Cvitkovic, M., and Karnin, Z. (2020). TabTransformer: Tabular Data Modeling Using Contextual Embeddings. arXiv:2012.06678.

Lundberg, S. M. and Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS, 30, 4765-4774.

Menon, A. K., Jayasumana, S., Rawat, A. S., Jain, H., Veit, A., and Kumar, S. (2021). Long-tail Learning via Logit Adjustment. ICLR.

Ren, J., Yu, C., Sheng, S., Ma, X., Zhao, H., Yi, S., and Li, H. (2020). Balanced Meta-Softmax for Long-Tailed Visual Recognition. NeurIPS, 33, 4175-4186.

Sharafaldin, I., Lashkari, A. H., and Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. ICISSP, 108-116.

Tavallaee, M., Bagheri, E., Lu, W., and Ghorbani, A. A. (2009). A Detailed Analysis of the KDD CUP 99 Data Set. CISDA, 1-6.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention Is All You Need. NeurIPS, 30, 5998-6008.

Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., and Venkatraman, S. (2019). Deep Learning Approach for Intelligent Intrusion Detection System. IEEE Access, 7, 41525-41550.

Yin, C., Zhu, Y., Fei, J., and He, X. (2017). A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks. IEEE Access, 5, 21954-21961.